QA Engineer Take-Home Assessment

Overview

We are building a robust ecosystem consisting of a web-based CAPI (Computer-Assisted Personal Interviewing) Survey Builder and an offline-first Android application used by field workers in remote areas.

Our users operate in challenging environments: budget phones (4GB RAM), patchy 2G networks, low device storage, and low literacy levels. The data they collect is critical. We need a QA Engineer who can think deeply about complex edge cases, automate smartly, and establish processes from the ground up.

Note: This is an in-depth, 4 Part hands-on take-home assignment. We expect this to take around 4-5 hours.

Reference Attachment Available

1.Android App Specification.html
2.Survey Builder Tool Capabilities.html
3.Sample Survey Schema.json

NOTE: Read all these reference documents carefully.

Required Submission

Please submit your work via a **GitHub Repository** link containing:

1. **Automated Test Scripts:** Your hands-on automation code (e.g., Postman collections, Cypress/Playwright scripts, etc.).
2. **Test Cases/Plans:** Documented test matrices/plans in CSV, Excel, or Markdown format.
3. `README.md`: A comprehensive README answering the scenario-based questions and explaining your setup, thought process, and tooling choices.

Part 1: Hands-On Technical Inspection & Automation

Scenario A: The Offline Sync & Data Collision (API & Mobile)

Context: Two field workers, Ankit and Pooja, are surveying the same village completely offline.

1. At 10:00 AM, Ankit registers "Govt Primary School" on his device, fills a survey, and his phone assigns it a local ID (e.g., `School_1`).
2. At 11:00 AM, Pooja visits the exact same school, registers it on her device, fills a survey, and her phone assigns it a local ID (e.g., `School_1`).
3. At 5:00 PM, they both reach a town with internet, and the app automatically syncs their data to the server at the exact same time via a background process.
4. When 50,000 users do this simultaneously, it essentially DDoSs our backend.

Your Task:

1. **API Automation (Hands-on):** Write a mock API test script (using Postman, RestAssured, Cypress, or your preferred tool) that simulates hitting a `POST /sync-survey` endpoint with a JSON payload. How would you write tests to ensure the API correctly handles duplicate local IDs without overwriting data? Include the script in your repo.
2. **Strategy (`README.md`):** Explain how you would test the Android app's background sync behavior under poor network conditions (e.g., network dropping exactly at 99% of a photo upload). What tools (e.g., Charles Proxy, ADB) would you use, and how?

Scenario B: Dynamic Form Rendering & Regression (Web & Mobile)

Context: Our web-based Survey Builder allows admins to create forms with 20+ input types, dynamic skip logic (visibility based on previous answers), and infinite repeating rosters (e.g., asking 5 questions for every household member). The mobile app downloads this complex JSON schema and renders the UI dynamically. Currently, when developers add a new feature to the builder, it often breaks existing form logic in the mobile app.

Attachment: “3.Sample Survey Schema.json”

Your Task:

1. **Web Automation (Hands-on):** Provide a sample automated test script (using Cypress, Playwright, or Selenium) that verifies a dynamic skip logic rule on a web form (e.g., If "Do you have a hobby?" = "Yes", assert that the "What is your hobby?" text field becomes visible).
2. **Strategy (`README.md`):** How do you design a regression test suite for an app where screens aren't static, but generated dynamically from JSON? What parts would you automate, and what parts would you leave for manual exploratory testing?

Scenario C: Offline Sync & State Management (Android WorkManager & Room)

Context: The Android app saves survey data locally in a Room database and uses `WorkManager` to trigger a background sync when internet connectivity is restored. We need to verify data integrity before, during, and after this sync process.

Your Task:

1. **Technical Inspection Strategy (`README.md`):** Imagine you are provided with the release APK. Write down the exact steps and tooling you would use to:
    - **Extract and inspect** the local Room SQLite database from an unrooted test device to verify an offline survey was saved correctly before syncing.
    - **Force** the background `WorkManager` sync job to execute immediately via ADB for testing purposes.
    - **Simulate a failure** by intercepting the network request (e.g., using Charles Proxy/Proxyman) and returning a `500 Internal Server Error`. How do you verify the app correctly schedules a retry without losing the local data?

Part 2: Complex Scenarios & Edge Cases

Please answer the following questions in your `README.md`.

Scenario D: The Edge-Case Device Constraints

Context: Our field workers are using 4GB RAM phones. We are getting reports that after watching 3-4 offline video training modules, the app suddenly crashes (ANR - Application Not Responding). However, it works perfectly on our high-end office test devices.

- **Question:** Walk us through exactly how you would reproduce this issue, debug it, and prove the root cause to a developer. What specific ADB commands, profilers, or device manipulation techniques would you use to simulate low storage and memory limits?

Scenario E: The HFC (High-Frequency Checks) Engine

Context: The Survey Builder includes an HFC engine that runs on the mobile device. It uses cross-question logic to ensure data quality (e.g.,
`if Age < 15 AND Education = 'College', Action = 'reask_and_flag'`

`OR`

`if Sleep Well = Yes AND Sleep hours < 6, Action = ‘reask_and_flag’`).

- **Question:** Design a brief test plan (can be a markdown table) for testing the HFC engine itself. Include positive, negative, and edge-case scenarios to ensure it blocks invalid data when necessary but doesn't trap a user in an infinite loop.

Part 3: QA Process & AI Integration

Please answer the following in your `README.md`.

1. **Process Creation:** You are joining as our first dedicated QA Engineer. Developers currently test their own code and push directly to the Play Store. Walk us through your first 30 days. How do you implement a structured QA process (entry/exit criteria, staging environments) without slowing down the fast-paced development team?
2. **Leveraging AI:** How do you currently use AI tools (like ChatGPT, GitHub Copilot, Gemini, etc.) in your day-to-day QA workflow? Give a specific example of how AI could help you write tests for the complex dynamic schemas mentioned in Scenario B.
3. **Automation Tooling:** With limited time, how do you decide what *should* be automated versus what *should not* be automated? Why did you choose the specific tools you used for Part 1 of this assignment?

Part 4: UI/UX & User-Centric QA

Please answer the following in your `README.md`.

Context: Our end-users are field workers in rural areas. They often have limited reading skills, use small-screen budget devices in bright sunlight, and rely heavily on visual cues (colors, icons). A feature might pass all functional tests but fail completely in the field because the text is too dense, the contrast is too low, or touch targets are too small.

Your Task:

1. **Scenario Evaluation:** A developer submits a new "Training Video Hub" screen for QA. It features a standard video player, a block of 3 paragraphs describing the video content, and a standard-sized "Next" button at the very bottom of a scrollable page.
2. **Question:** As a QA Engineer:
    - how would you evaluate this screen beyond basic functional testing?
    - List at least 5 specific UX/UI issues or rendering risks you would flag based on our target demographic (low literacy, budget phones).
    - How do you systematically test for these issues, and how do you constructively communicate these "non-functional" bugs to the design and development teams?